

TORNEO-UNI · FASE 2

ReservaEspacios

Aplicacion web para la reserva de espacios

Caso de estudio A · Reservas de espacios (salas, escritorios y canchas)

Integrantes:

Victor Rafael Arevalo Sierra

Gabriela Abigail Diaz Rivera

Victor Manuel Mira Hernandez

Diego Marcelo Rivera Lopez

Luis Jose Rodriguez Centeno

Repositorio: <https://github.com/Cent0403/ReservaEspacios> (etiqueta `v2.0`)

1 de octubre de 2026

2. Cambios respecto a la propuesta de la Fase 1

El modelo de clases de la Fase 1 se mantuvo como base, pero la evolución hacia una aplicación web exige ajustes que se documentan a continuación.

Area	Fase 1 (propuesta)	Fase 2 (implementado)
Tipos de espacio	Entidades de ejemplo de otra línea: <code>CanchaFutbol</code> , <code>CanchaTenis</code> y <code>Piscina</code> .	Se ajusta al Caso A asignado: Sala (sala de reunión), Escritorio y Cancha (deporte), que son los tipos que exige la rúbrica.
Interfaz Reservable	Incluía <code>hayDisponibilidad()</code> junto a <code>calcularCosto()</code> .	La disponibilidad ya no es una pregunta al objeto: dos reservas se traslapan o no según los datos actuales de la base, se consulta con SQL (<code>hora_inicio < fin AND hora_fin > inicio</code>). La interfaz quedó reducida a <code>calcularCosto(float \$horas, bool \$horarioPico)</code> , que es la parte genuinamente polimórfica.
Namespace de servicios	<code>App\Servicios\ArchivoReservas</code> .	Se alineó a los nombres en inglés del autoloading PSR-4: <code>App\Services\ArchivoReservas</code> (commits de refactor).
Persistencia	Arreglos en <code>main.php</code> y archivo JSON.	MySQL/MariaDB con PDO y consultas preparadas; el JSON de la Fase 1 se conserva solo como demostración de escritorio (<code>php main.php</code>).
Capas nuevas	No existían.	Repositorios, fábrica, validador, servicio de imágenes, helpers de sesión/CSRF, layout y vistas, páginas de <code>public/</code> y pruebas end-to-end.
Recargos y "dato calculado"	Tarifas fijas por tipo en la demo.	Recargo de horario pico (desde las 17:00): Sala 1,20 · Escritorio 1,10 · Cancha 1,25, y un <i>dato calculado</i> distinto por tipo (2 h, 4 h y 1 h) que hace visible el polimorfismo en listados, fichas y reporte.

Lo que no cambió: la jerarquía `Espacio` abstracta con métodos polimórficos, la entidad `Reserva` compuesta por un `Espacio`, las interfaces `Reservable` y `Exportable`, y la prueba de humo de la Fase 1 (sigue funcionando con `php tests/smoke.php`).

Respuesta a la reflexión 1 · ¿Por qué tabla única?

Se eligió la **tabla única con columna tipo** (Single Table Inheritance) porque el listado, el reporte y el panel necesitan ordenar y contar los tres tipos juntos sin hacer `JOIN`s, y porque la aplicación siempre recupera espacios completos: una sola consulta `SELECT *` trae todo y la fábrica decide qué subclase instanciar. La desventaja de la opción que no se eligió (tabla base + tabla por tipo) es que exige un `JOIN` por cada lectura y tres tablas que mantener en cada consulta de listado; además, al ser la base de datos pequeña (un centro deportivo), la normalización extra no aportaba y sí complejaba los `CHECK` y la reutilización de código.

3. Modelo relacional y estrategia de herencia

Diagrama entidad-relación

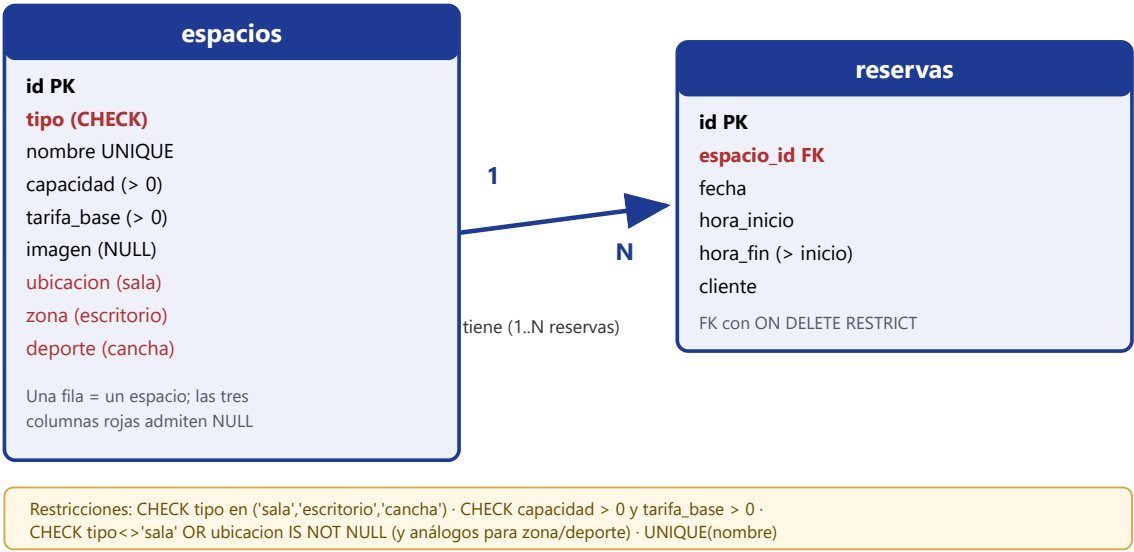


Figura 1. Diagrama entidad-relación del Caso A.

Estrategia elegida: tabla única con columna tipo

`espacios` guarda los campos comunes de las tres subclases y tres columnas específicas que solo se llenan cuando el `tipo` corresponde (si no, quedan en `NULL`, reforzado con `CHECK`). La lógica de decidir qué clase se construye a partir de una fila vive únicamente en `EspacioFactory::desdeFila()`.

`reservas` se relaciona con `espacios` mediante `espacio_id` con `ON DELETE RESTRICT`: no se puede borrar un espacio que tenga reservas (el sistema lo avisa en lugar de borrar en cascada). Además, el horario tiene un `CHECK` (`hora_fin > hora_inicio`) y la prohibición de traslapes se verifica en la capa de acceso a datos con una consulta que compara intervalos.

4. Diagrama de clases actualizado

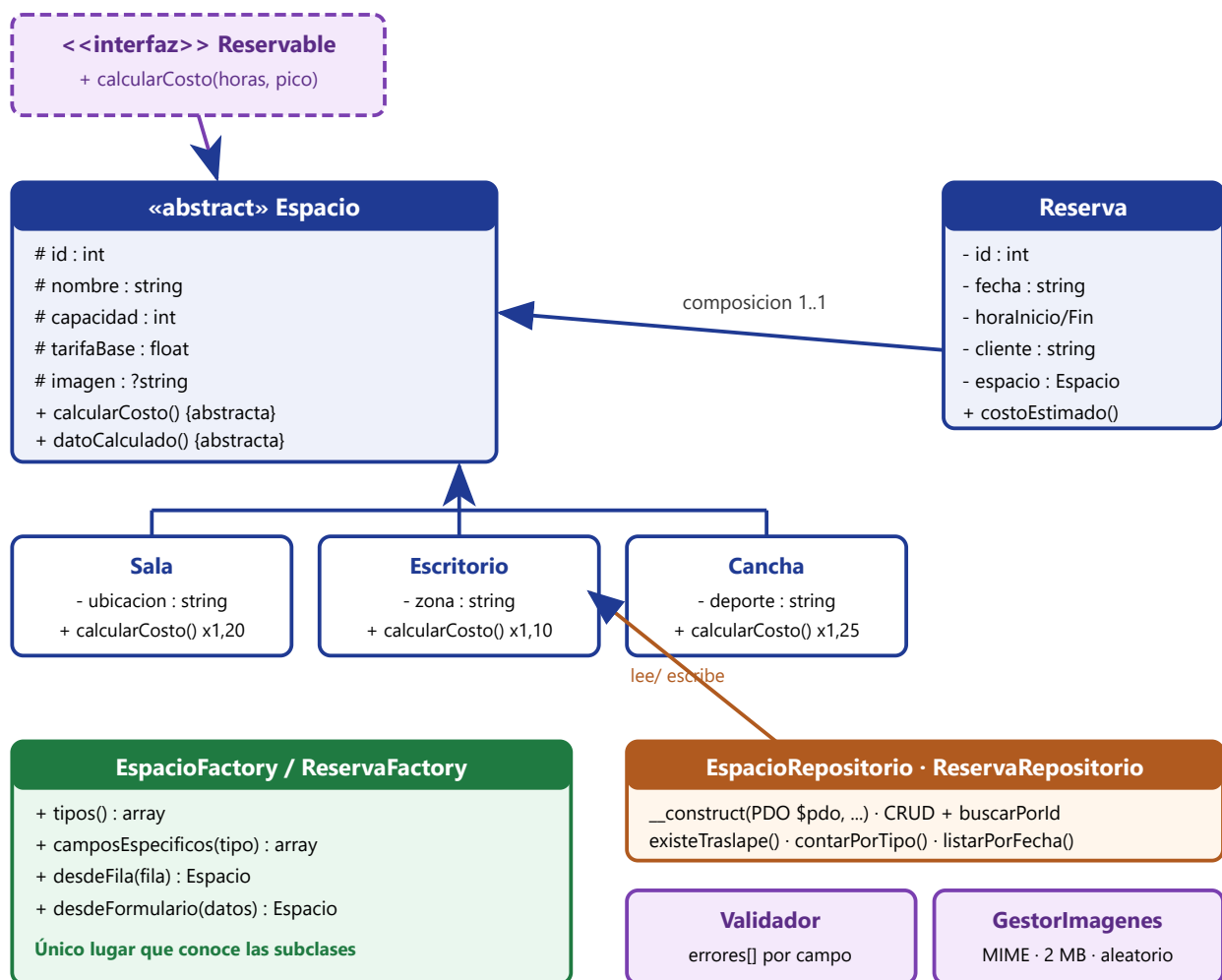


Figura 2. Diagrama de clases de la Fase 2: dominio, fábrica, repositorios y servicios auxiliares.

5. Correspondencia clases ↔ tablas

Clase (namespace)	Tabla	Columnas	Observaciones
App\Espacios\Espacio (abstracta)	espacios	id, tipo, nombre, capacidad, tarifa_base, imagen	Una fila por objeto; tipo guarda la subclase concreta.
App\Espacios\Sala	espacios	comunes + ubicacion	ubicacion NOT NULL cuando tipo='sala' (CHECK).
App\Espacios\Escritorio	espacios	comunes + zona	zona NOT NULL cuando tipo='escritorio' (CHECK).
App\Espacios\Cancha	espacios	comunes + deporte	deporte NOT NULL cuando tipo='cancha' (CHECK).
App\Entidades\Reserva	reservas	id, espacio_id, fecha, hora_inicio, hora_fin, cliente	espacio_id es la FK a espacios.id; la propiedad en memoria es un objeto Espacio (composición).
App\factories\EspacioFactory	—	Une tipo → clase en un solo mapa	No persiste: solo traduce fila/formulario a objeto y viceversa.
App\Validation\Validador	—	—	Acumula errores por campo antes de tocar la base.
App\Services\GestorImagenes	espacios.imagen	Guarda solo el nombre aleatorio generado (p. ej. 5b37c6b5....png) en public/uploads/	Nunca se guarda el nombre original ni la ruta completa.
App\Database\Conexion	—	—	Devuelve la instancia PDO (inyección de dependencias).

6. Tabla de validaciones

Campo	Regla en el cliente	Regla en el servidor	Mensaje mostrado
Nombre (espacio)	required y maxlength=80; JS marca aria-invalid.	requerido() + maximo(80) en Validador.	«El campo Nombre es obligatorio.»
Capacidad	type="number" min="1" required.	requerido() + enteroMayorQue(0).	«El campo Capacidad debe ser mayor que 0.»
Tarifa base	type="number" min="0.01" step="0.01" required.	requerido() + decimalMayorQue(0).	«El campo Tarifa base debe ser mayor que 0.»
Tipo	Selector con las tres opciones; al cambiarlo se muestran/ocultan los campos propios y se activa required.	unoDe() contra EspacioFactory::tipos().	«El campo Tipo de espacio no es valido.»
Ubicación / Zona / Deporte	Solo visibles para su tipo, con required solo cuando el tipo está activo.	camposEspecificos() según el tipo elegido.	«El campo Ubicacion (edificio y salon) es obligatorio.»

Campo	Regla en el cliente	Regla en el servidor	Mensaje mostrado
Imagen	<code>accept="image/*"</code> y vista previa en JS.	Error de carga PHP, tamaño ≤ 2 MB y MIME real con <code>finfo</code> (JPG/PNG/WEBP).	«Formato no permitido. Use una imagen JPG, PNG o WEBP.» / «...supera el tamaño máximo permitido de 2 MB.»
Cliente (reserva)	<code>required</code> , <code>maxlength=80</code> .	<code>requerido()</code> + <code>maximo(80)</code> .	«El campo Cliente es obligatorio.»
Fecha y horas	<code>type="date"</code> , <code>type="time"</code> ; JS exige <code>fin > inicio</code> .	Formato YYYY-MM-DD válido con <code>checkdate()</code> ; <code>hora_fin > hora_inicio</code> (también CHECK en BD).	«El campo Hora fin debe ser posterior al campo Hora inicio.»
Traslape	No aplica (depende de los datos guardados).	<code>ReservaRepositorio::existeTraslape()</code> : compara intervalos del mismo espacio y fecha.	«El espacio ya tiene una reserva que se traslapa con ese horario.»
Token CSRF	Campo oculto en cada formulario.	<code>csrf_verificar()</code> en todos los POST; sin token válido no se procesa nada.	«La sesión expiró o el formulario no es válido. Intentalo de nuevo.»

7. Evidencias

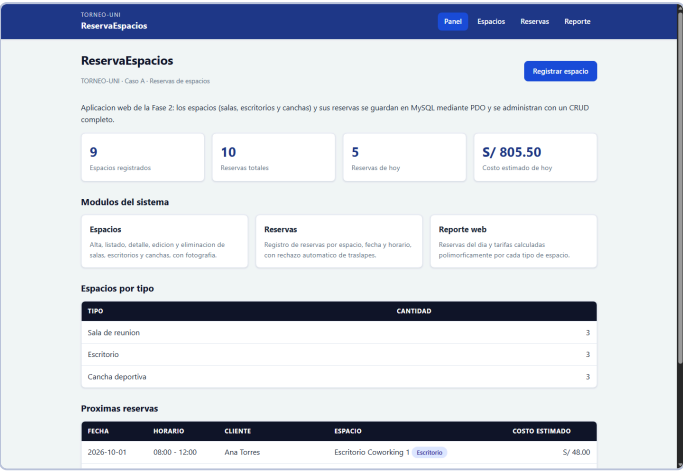


Figura 3. Panel inicial: totales, accesos y reservas próximas (página 1 de 8).

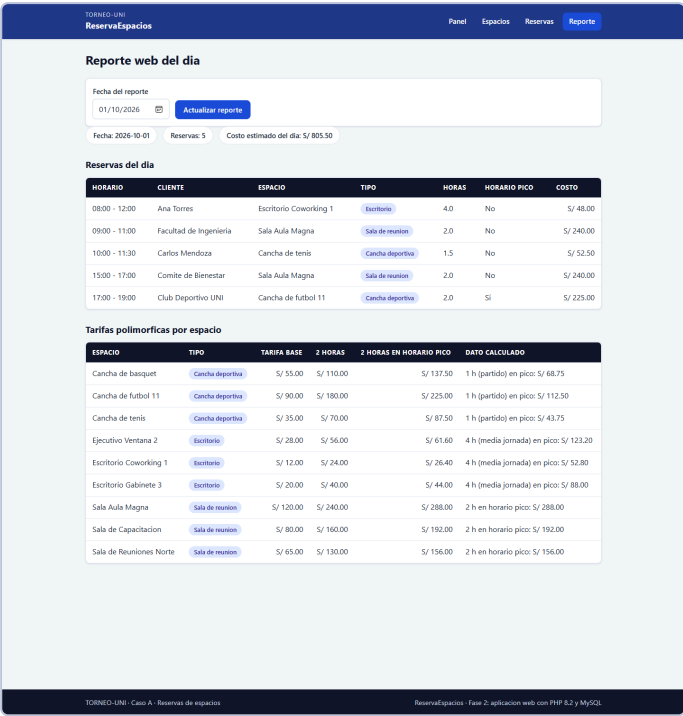


Figura 4. Reporte web del día con costos polimórficos (página 8 de 8).

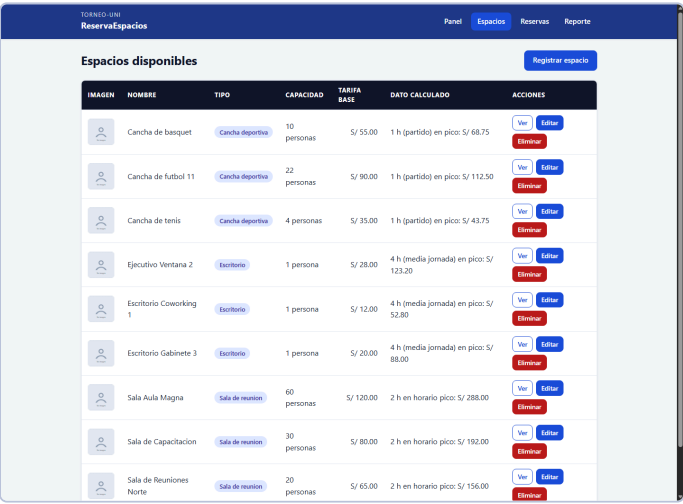


Figura 5. Listado de espacios (CRUD: Read) con imagen por defecto, insignia de tipo y dato calculado.

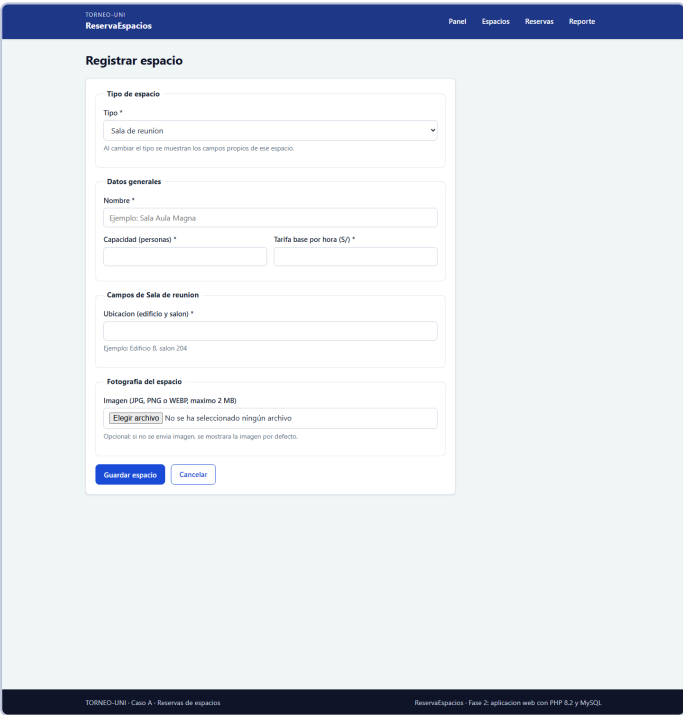


Figura 6. Formulario de alta con fieldsets por tipo, campos propios e imagen opcional.

7. Evidencias (continuación)

Registrar espacio

Tipo de espacio

Tipo *

Sala de reunion

Al cambiar el tipo se muestran los campos propios de ese espacio.

Datos generales

Nombre *

Ejemplo: Sala Aula Magna

El campo Nombre es obligatorio.

Capacidad (personas) *

0

El campo Capacidad debe ser mayor que 0.

Tarifa base por hora (S/) *

-5

El campo Tarifa base debe ser mayor que 0.

Campos de Sala de reunion

Ubicacion (edificio y salon) *

El campo Ubicacion (edificio y salon) es obligatorio.

Ejemplo: Edificio B, salon 204

Fotografia del espacio

Imagen (JPG, PNG o WEBP maximo 2 MB)

Elige archivo

No se ha seleccionado ningún archivo.

Opcional: si no se envia imagen, se mostrara la imagen por defecto.

Guardar espacio Cancelar

Figura 7. Prueba de validación en el servidor: POST real con datos inválidos; aparecen los cuatro errores por campo, se conservan los valores (0 y -5) y se aplican las clases `.campo-error`.

Registrar reserva

Datos de la reserva

Espacio *

Sala Aula Magna (Sala de reunion)

Cliente *

Cliente de prueba

Fecha *

01/10/2026

Hora de inicio *

09:30

El espacio ya tiene una reserva que se traslapa con este horario.

Hora de fin *

10:30

La hora de fin debe ser posterior al inicio y el horario no puede trasladarse con otra reserva del mismo espacio y día.

Guardar reserva Cancelar

Figura 8. Reserva traslapada rechazada por el servidor junto al campo Hora de inicio (validación propia del Caso A).

FECHA	HORARIO	CLIENTE	ESPACIO	HORAS	COSTO ESTIMADO	ACCIONES
2026-10-03	16:00 - 18:00	Selección Universitaria	Cancha de basquet Cancha deportiva	2.0	S/ 110.00	Ver Editar Eliminar
2026-10-02	09:00 - 13:00	Lucía Fernández	Escritorio Gabinete 3 Escritorio	4.0	S/ 80.00	Ver Editar Eliminar
2026-10-02	14:00 - 16:00	Departamento de Sistemas	Sala de Reuniones Norte Sala de reunion	2.0	S/ 130.00	Ver Editar Eliminar
2026-10-01	08:00 - 12:00	Ana Torres	Escritorio Coworking 1 Escritorio	4.0	S/ 48.00	Ver Editar Eliminar
2026-10-01	09:00 - 11:00	Facultad de Ingeniería	Sala Aula Magna Sala de reunion	2.0	S/ 240.00	Ver Editar Eliminar
2026-10-01	10:00 - 11:30	Carlos Mendoza	Cancha de tenis Cancha deportiva	1.5	S/ 32.50	Ver Editar Eliminar
2026-10-01	15:00 - 17:00	Comité de Bienestar	Sala Aula Magna Sala de reunion	2.0	S/ 240.00	Ver Editar Eliminar
2026-10-01	17:00 - 19:00	Club Deportivo UNI	Cancha de futbol 11 Cancha deportiva	2.0	S/ 225.00	Ver Editar Eliminar
2026-09-30	10:00 - 12:00	Seminario de Titulación	Sala de Capacitación Sala de reunion	2.0	S/ 160.00	Ver Editar Eliminar
2026-09-30	14:00 - 18:00	Diego Ramirez	Ejecutivo Ventana 2 Escritorio	4.0	S/ 112.00	Ver Editar Eliminar

Figura 9. Listado de la entidad relacionada Reserva con horas y costo estimado.

Confirmar eliminación de espacio

Vas a eliminar Sala Aula Magna. Esta acción no se puede deshacer y también borrará la imagen asociada.

Tipo

Sala de reunion

Capacidad

60 personas

Tarifa base

S/ 120.00 por hora

Imagen

Imagen por defecto

Ubicación (edificio y salon)

Edificio principal, planta baja

Si, eliminar Cancelar

Figura 10. Eliminación con confirmación: el GET solo muestra la pantalla de confirmación; el borrado ocurre por POST con CSRF.

7. Evidencias (continuación)

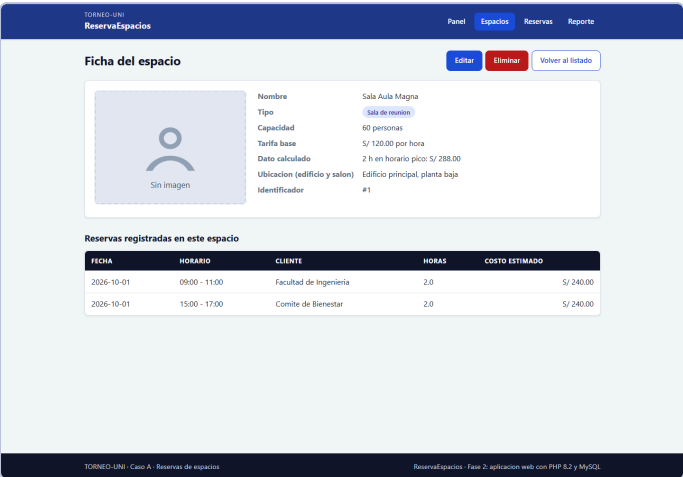


Figura 11. Ficha (detalle) del espacio con datos específicos de su tipo y costo calculado.

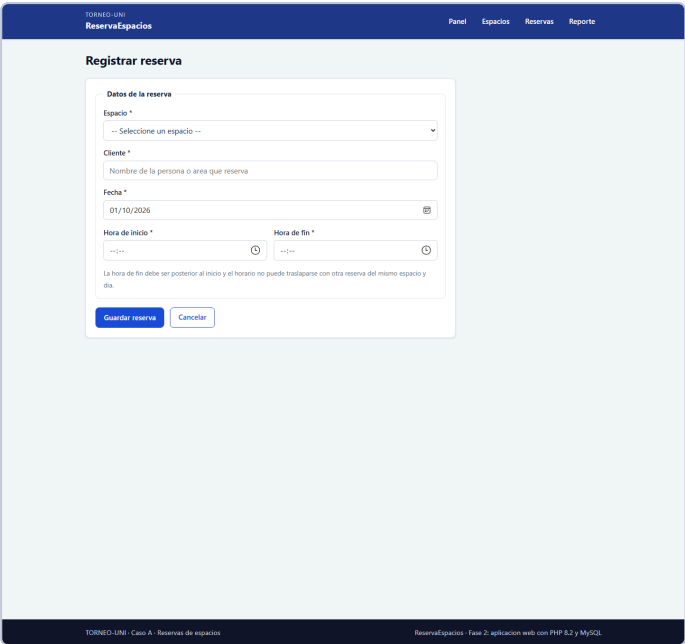


Figura 12. Formulario de reserva (entidad relacionada) con ayudas de horario.

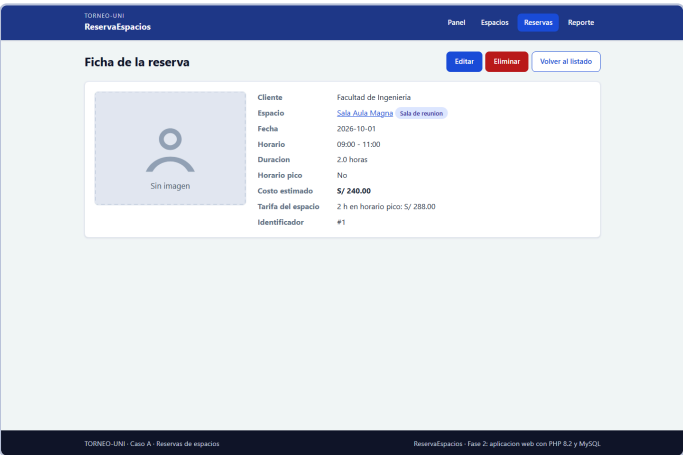


Figura 13. Ficha de la reserva con costo estimado polimórfico y marca de horario pico.

Prueba automatizada: los scripts `tests/e2e/web-espacios.ps1` y `tests/e2e/web-reservas-imagenes.ps1` ejecutan más de 100 comprobaciones reales contra el servidor (HTTP 200, HTML5, CSRF válido e inválido, PRG, errores por campo, subida y reemplazo de imagen, rechazo de MIME falso y de archivos > 2 MB, eliminación bloqueada por reservas, traslapes, reporte por fecha y escape de salida) y terminan con `FALLOS=0`.

8. Justificación de los cuatro pilares en la Fase 2

Pilar	Dónde se mantiene en la versión web	Evidencia en el código
Abstracción	<code>Espacio</code> sigue siendo una clase abstracta que define el contrato de cálculo sin conocer detalles de cada tipo; las interfaces <code>Reservable</code> y <code>Exportable</code> delimitan capacidades.	<code>src/Espacios/Espacio.php</code> (métodos abstractos <code>etiquetaTipo()</code> , <code>datoCalculado()</code>); <code>[ABSTRACCION]</code> en su docblock.
Encapsulamiento	Propiedades <code>private/protected</code> con getters, setters que validan y <code>aFila()</code> declarado <code>final</code> : nadie altera el estado por fuera.	<code>src/Espacios/Espacio.php</code> , <code>src/Entidades/Reserva.php</code> ; <code>[ENCAPSULAMIENTO]</code> .
Herencia	<code>Sala</code> , <code>Escritorio</code> y <code>Cancha</code> extienden <code>Espacio</code> , añaden su atributo propio y reutilizan el constructor con <code>parent::__construct()</code> .	<code>src/Espacios/{Sala,Escritorio,Cancha}.php</code> ; <code>[HERENCIA]</code> .
Polimorfismo	Los listados, las fichas y el reporte llaman <code>calcularCosto()</code> y <code>datoCalculado()</code> sin preguntar de qué tipo es cada espacio; la única decisión por tipo vive en la fábrica.	<code>views/espacios/listado.php</code> , <code>views/reporte.php</code> , <code>public/reporte.php</code> ; <code>[POLIMORFISMO]</code> y <code>[FABRICA]</code> .

Respuestas breves a las demás preguntas de reflexión

2 · ¿Por qué solo la fábrica conoce las clases concretas? Porque es el único punto que debe cambiar cuando aparece un tipo nuevo (por ejemplo, «Piscina»): basta con crear la clase, agregar la etiqueta/tipo y una entrada en el mapa de `EspacioFactory`. El resto del sistema (vistas, repositorios, reporte) recorre `EspacioFactory::tipos()` y `camposEspecificos()`, así que no se toca. Si la decisión se dispersara en las vistas, habría que buscar `instanceof` por todo el código.

3 · ¿Qué ataque evitan las consultas preparadas? La inyección SQL. Entrada maliciosa: `' OR '1'='1` como nombre de cliente, o `10; DROP TABLE reservas;--` en un campo numérico. Con `prepare()/execute()` el valor viaja como dato en un protocolo separado del código SQL, así que se interpreta literalmente: no altera el `WHERE` ni ejecuta sentencias extra. Prueba: el script e2e envía `<script>alert(1)</script>` como nombre y el sistema lo guarda/escapa sin ejecutarlo.

4 · ¿Por qué validar también en el servidor? Porque el navegador es solo cortesía: cualquiera puede desactivar JavaScript, usar un cliente distinto (curl, un bot) o reenviar el formulario a mano. El `Validador` y los repositorios son la barrera real; el e2e lo demuestra enviando POSTs sin pasar por el formulario.

5 · ¿Por qué no confiar en la extensión ni en el nombre original? Porque el nombre lo elige quien sube el archivo (`foto.php.jpg`, caracteres de ruta, etc.). Se valida el MIME real con `finfo` sobre el contenido y se guarda con un nombre aleatorio generado por el sistema (`bin2hex(random_bytes(16))`), con `basename()` al borrar.

6 · ¿Ventajas de los repositorios? El SQL vive en un solo lugar (dos clases), las páginas de `public/` solo piden operaciones de alto nivel, es fácil reutilizar y probar (el e2e ejercita los mismos métodos), y cambiar la persistencia o añadir una consulta no obliga a tocar las vistas.

9. Distribución del trabajo

El reparto acordado por el equipo y su reflejo en el historial de commits (los aportes se confirman con `git shortlog -sne`):

Integrante	Areas a cargo	Estado en el historial
Luis José Rodríguez Centeno (Centeno)	Base de la Fase 1: modelo de clases de la propuesta, autoloading PSR-4 con Composer, datos de arranque, prueba de humo (<code>tests/smoke.php</code>) y documentación inicial.	Commits del 2026-08-20 (Fase 1).
Victor Rafael Arévalo Sierra (Vic2910)	Base de datos y scripts; repositorios con PDO; fábricas; validador; servicio de imágenes; CRUD completo de espacios y reservas; layout, vistas y CSS; reporte polimórfico; pruebas end-to-end; README e informe.	Commits <code>feat/fix/test/docs</code> de la Fase 2 (2026-10-01 en adelante).
Gabriela Abigail Díaz Rivera	Validación en cliente y accesibilidad: mensajes de <code>public/js/formulario.js</code> , estados de error, etiquetas y <code>aria-*</code> en los formularios.	Tareas detalladas en <code>docs/tareas-por-integrante.md</code> (commits pendientes de subir con su usuario).
Victor Manuel Mira Hernández	Contenido de las páginas y datos de prueba: textos de ayuda, vacíos, ampliación del <code>seed.sql</code> y revisión de las fichas.	Tareas detalladas en <code>docs/tareas-por-integrante.md</code> (commits pendientes de subir con su usuario).
Diego Marcelo Rivera López	Reporte y utilidades: exportación del reporte, filtros adicionales y revisión de las pruebas.	Tareas detalladas en <code>docs/tareas-por-integrante.md</code> (commits pendientes de subir con su usuario).

Cada integrante sube sus commits con su propia identidad de Git y en días distintos (la rúbrica pide 5 commits por integrante en 3 o más días). El historial no se modifica retroactivamente: los aportes nuevos se agregan por encima de la etiqueta `v2.0`.

10. Conclusiones y dificultades

Conclusiones

- La POO “no se rompe” al pasar a web: las mismas clases de la Fase 1 alimentan vistas y reportes; lo que cambió fue dónde viven los datos (BD) y quién los pide (repositorios).
- Concentrar la decisión de tipo en una sola fábrica hizo posible que ninguna vista contenga `instanceof` ni `switch` por tipo: agregar un cuarto tipo no toca las vistas.
- PRG + mensajes flash + CSRF eliminaron los reenvíos duplicados y la duda sobre si una acción se ejecutó.
- Las pruebas end-to-end (más de 100 comprobaciones) detectaron bugs reales durante el desarrollo, por ejemplo un panel que imprimía «Array» y un archivo de imagen que quedaba huérfano en Windows al fallar un INSERT.

Dificultades encontradas

- Puertos de la base:** el equipo convivió con dos servidores MySQL; se estandarizó la conexión a la MariaDB de XAMPP que atiende PHPMyAdmin (puerto 3306) y se dejó documentado en `config/config.example.php`.
- Borrado de imágenes en Windows:** `unlink()` puede fallar si el antivirus bloquea el archivo recién escrito; se añadieron reintentos en `GestorImagenes::eliminar()`.

- **Idempotencia de las pruebas:** las primeras corridas dejaban datos propios que rompían la siguiente; se añadió limpieza previa para poder ejecutar la suite cuantas veces haga falta.
- **Disponibilidad vs. polimorfismo:** decidir qué parte de la interfaz `Reservable` dejó de pertenecer al objeto (disponibilidad) y qué se mantuvo (costo) exigió reescribir el contrato y la documentación de la Fase 1.